

Agent-Oriented Architecture

A practical field guide for designing and governing agent systems

From the workshop to your estate

Pass	Session job	What changes
1. See the shape	Make AOA visible through small concrete demos	Small workflow
2. Name the shape	Inspect the running parts in AOA vocabulary	Running system
3. Recognise the shape	See the architecture in the wider landscape	Real landscape
4. Apply the shape	Turn the architecture into a contextual adoption loop	Adoption loop

This guide is the take-away version of the course: key diagrams, concise explanations, templates, diagnostic questions, and the exercises you can rerun from the companion `aoa-course` repository.

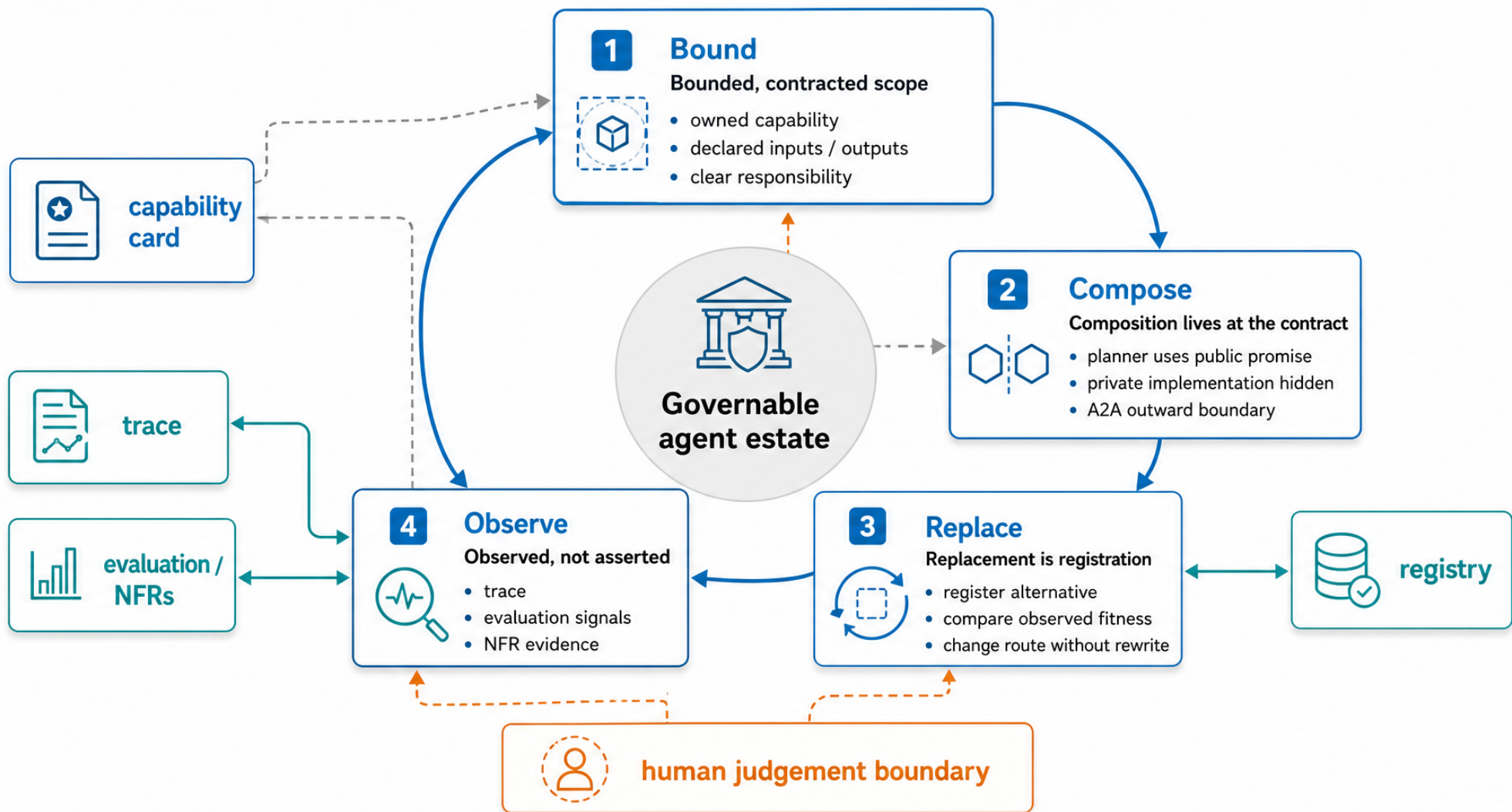
The lifecycle, eight verbs

`describe → discover → compose → execute → observe → evaluate → replace → govern`

Same eight verbs in every session. Different depth each pass.

The four AOA principles (full detail overleaf)

`bounded, contracted scope · composition lives at the contract · replacement is registration · observed, not asserted`



Principles are useful when they change architecture decisions.

The four AOA principles

The load-bearing architectural commitments. Each one is architectural (not operational), falsifiable, independent of the others, atomic, and earns its rent.

1. Bounded, contracted scope

Each Agentic Unit owns a narrow capability with declared inputs, outputs, constraints, and responsibility — a public contract on a capability card. Scope is bounded; unbounded AUs are not AUs.

Decompose your agent estate into units you can actually govern — not frameworks you have to maintain.

2. Composition lives at the contract

Work is composed by reading public capability cards through the registry, not by lines of code that hard-wire one agent to another. The planner reasons over declared promises and constraints at runtime.

Integration without picking a winner. Your existing platform bets stay in place; AOA is what makes them add up.

3. Replacement is registration, not a rewrite

Substituting one AU for another — different backend, model, vendor, or implementation — is a registry operation. Callers don't change; code doesn't change; the planner picks up the new candidate on the next selection.

Architected to outlive the model of the month. No more 18-month re-platforms when the next thing drops.

4. Observed, not asserted

A capability card is a claim; the trace is evidence. Once traffic flows, runtime measurements override the declarations. Selection uses what the system has measured, not what the AU claims. (The architectural reply to UDDI's failure mode.)

Run it like an estate, not like a pilot. Cost, latency, and quality become first-class signals — including something you can take to your CFO.

How to use this guide

This is not a transcript. It is a working reference for the first architecture conversation after the course.

Four reading paths

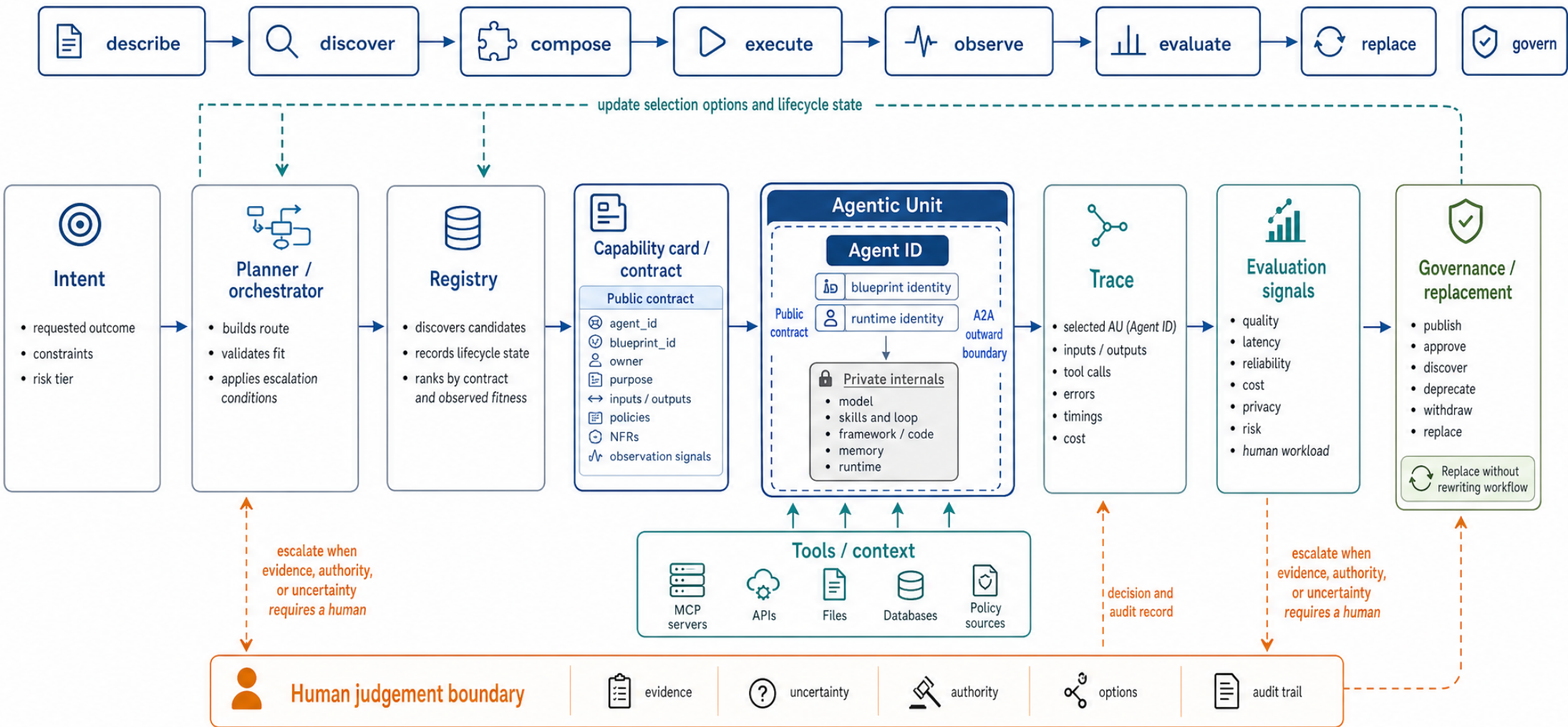
Your immediate question	Start here	Leave with
What is AOA?	Sessions 1 and 2	The eight control surfaces and four principles
How does this fit our stack?	Session 3	A framework/protocol/platform placement model
How do we avoid vendor lock-in?	Protocols, portability, and the EVE lab	A stable outward contract around replaceable internals
What should we do next?	Session 4 and the adoption canvas	A boundary, active constraint, next move, and owner

Three claim disciplines

1. **Architecture is not a product.** AOA is a set of responsibilities and constraints. Your platforms may discharge some of them.
2. **A card is a claim; a trace is evidence.** Production observations should eventually affect selection and replacement.
3. **Evidence is not permission.** Governance surfaces can support assurance; they cannot make legal or risk decisions by themselves.

Companion material

The `aoa-course` repository contains the runnable reference system, Docker profiles, pre-work, and lab instructions. The examples are deliberately small enough to inspect. Treat them as teaching code, not a production platform.



Governability comes from Agent ID, stable contracts, observable behaviour, explicit judgement boundaries, and replaceable capabilities.

Session 1 · See the shape

Make AOA visible through small concrete demos, then name the four principles.

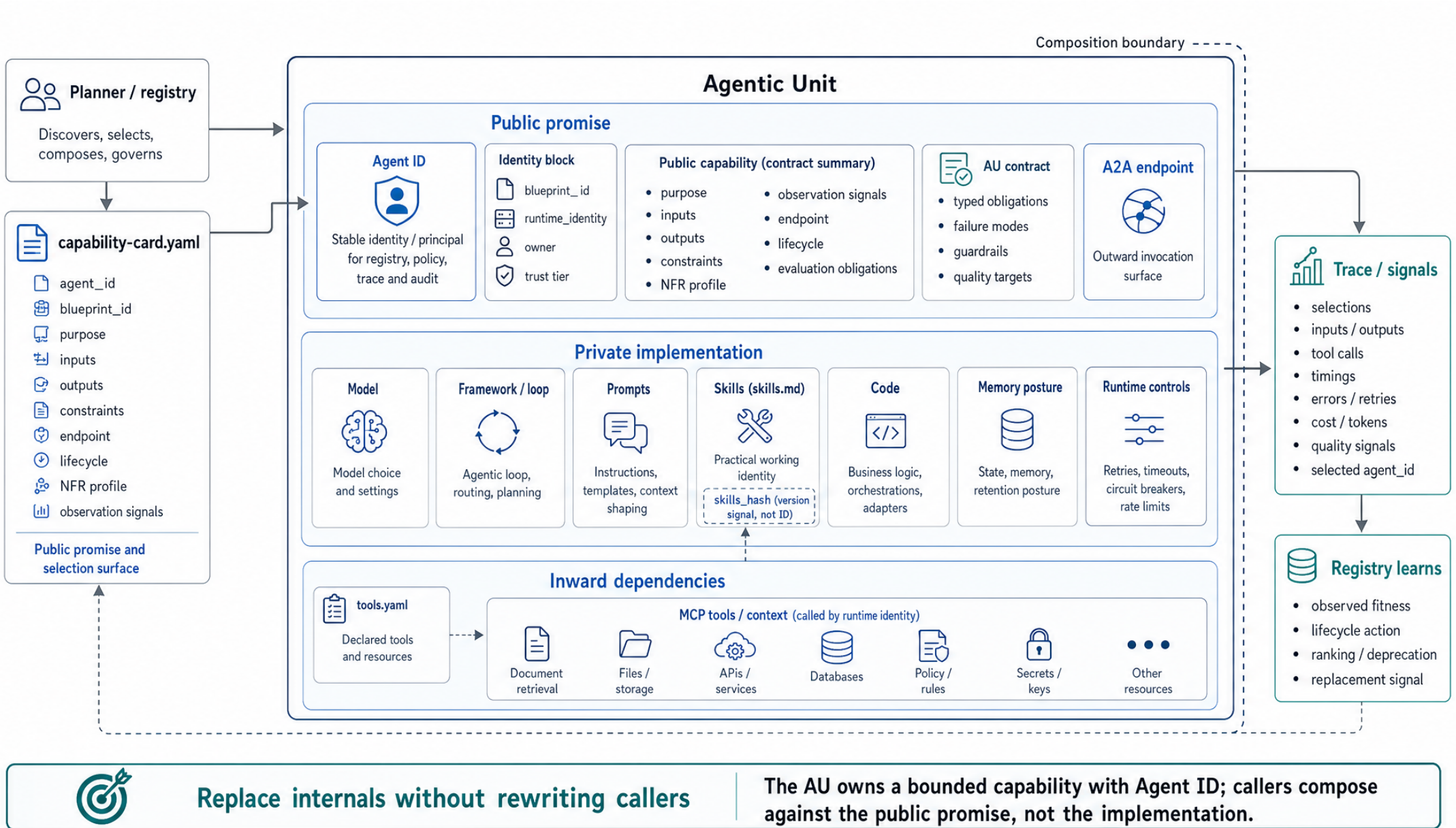
Key notes

- **The agentic inversion.** Workflow composition moves from a human in the loop to a planner reasoning over capability contracts. The unit of value isn't a clever agent — it's a fleet that can be discovered, composed, observed, and replaced.
- **Agent sprawl is the failure mode.** Without a shared catalogue, identity, and governance, duplicate capabilities accumulate, unowned agents drift, and shadow agents proliferate. AOA's principles exist to govern the fleet, not to fight individual agents.
- **Eight key components** make an agent system governable: *AU · capability card · registry · planner · tool boundary · trace · evaluation signal · human oversight point*.
- **Four principles** — bounded scope · composition at the contract · replacement is registration · observed, not asserted. They are architectural constraints, not features of a product.
- **The principles aren't theory.** The expense-report demo shows all four principles working: messy receipts become a policy-checked report through bounded, discovered, observable units — and a small local model carries each step. Local models swap in for frontier models without rewriting the workflow: *replacement is registration* in one demo moment.

Spot the eight key components

During the demo, see if you can spot each one. Pair up after the demo for two minutes and compare notes.

#	Component	What it is
1	Agentic Unit (AU)	a bounded capability — one job
2	Capability card	public contract — inputs / outputs
3	Registry	where cards live + are discoverable
4	Planner	composes work by reading cards
5	Tool boundary	where an AU calls a tool (MCP)
6	Trace	runtime record of what happened
7	Evaluation signal	observed quality — cost, latency, confidence
8	Human oversight point	where a human reviews / approves / escalates



Session 2 · Name the shape

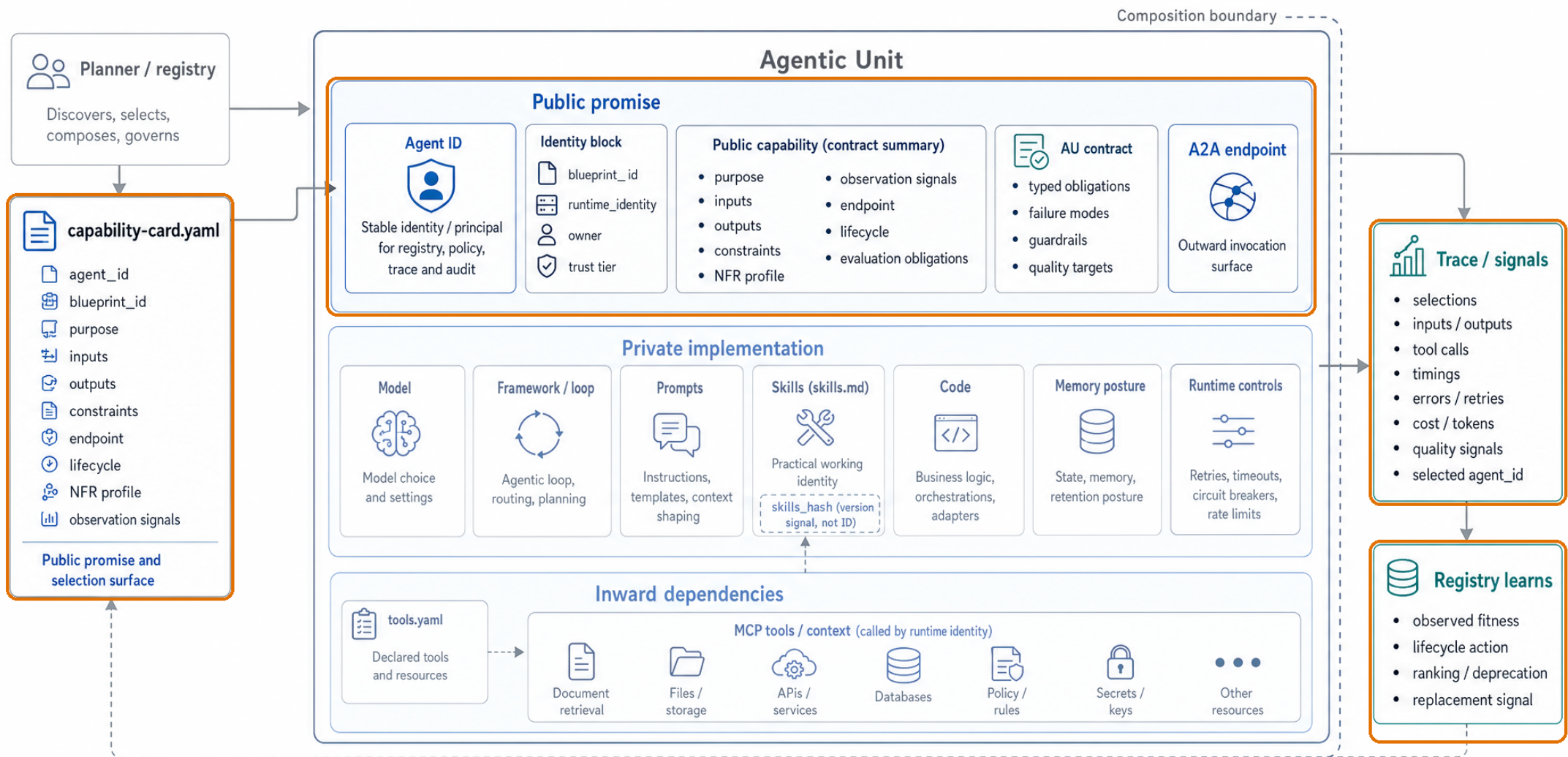
Deep dive on the AU. The control surfaces from Session 1 become inspectable, walkable, and governable.

Key notes

- **AU anatomy.** A public *capability card* (what) wraps private *internals* (how) — model, framework, skills, code, memory, runtime. **Agent ID** is the stable governed identity; hashes are integrity signals, not identity.
- **The capability card is the public promise.** Inputs · outputs · constraints · NFRs · owner · lifecycle state · evaluation evidence · human-oversight policy. The planner reasons over the card; consumers compose against the card.
- **Instructions are governed prose.** The behaviour file is `instructions.md` — markdown read by both humans and the model, bound to a specific capability card. Edits are first-class governance events — recorded as `skills_hash` (the hash keeps its historical name) so observations can be reconciled to the behaviour version that produced them.
- **Tools are not capabilities.** Both can be registered, but a tool is deterministic (MCP-style); an AU has model-driven behaviour. The registry holds both with different `kind`.
- **Trace makes responsibility walkable.** Intent → planner → registry → selected AU → tool calls → outputs → evaluation signals. Correlation IDs tie it together. Without a walkable trace, "observed, not asserted" is a slogan.
- **Silent recovery vs explicit escalation.** Silent recovery is permitted *inside* the contract and *visible in the trace*. Escalate at constraint boundaries — missing capability, irreversible side-effect, trust-tier breach.

The lab you did

You edited `system/agents/evaluator/capabilities/cv/instructions.md` in the running system, watched `skills_hash` re-register with no restart, and reran the workflow. To repeat at home: `./scripts/session1-up.sh`, Studio at `localhost:8080`. The stretch: swap the model provider in `.env` and watch `provenance.model` change behind the same card.



Replace internals without rewriting callers

The AU owns a bounded capability with Agent ID; callers compose against the public promise, not the implementation.

The capability card

Write the public promise before the private prompt.

Minimum useful shape

```
id: red-flags-review
version: 0.1.0
kind: au
purpose: Identify evidence-backed risks that require human review.
inputs:
  - {name: artefact, type: text, required: true}
outputs:
  - {name: findings, type: array}
constraints:
  - Cite the source passage for every finding.
  - Make no final employment decision.
evaluation_signals:
  - valid_output_shape
  - all_findings_cited
provenance:
  model: ${MODEL}
```

Questions the card must answer

Surface	Question
Purpose	What one business capability does this unit own?
Inputs and outputs	What can a caller rely on without knowing the implementation?
Authority	What may it decide, recommend, change, or never do?
Tools and data	What may it access, and under whose authority?
Oversight	Which constraint or side effect forces human review?
Evidence	Which signals would falsify its claims?
Lifecycle	Who owns, reviews, approves, deprecates, and replaces it?

Boundary test

An AU should have one dominant reason to change, a coherent authority boundary, independent evaluation, and a contract another team can use. If it is deterministic, it is probably a tool. If it only exists to pass data to the next prompt, it is probably a workflow step rather than an AU.

 Era (years)	 Contract	 Discovery	 Coordination	 Observation
 Distributed objects (1989)	IDL	OMG Trading Service	Activation Services	logs
 SOA / web services (2000)	WSDL	UDDI	BPEL / ESB	logs / SLAs
 Pragmatic REST (2000)	OpenAPI	(informal — lost)	application code	tracing
 Microservices (2011)	OpenAPI / protobuf	Eureka / Consul / K8s	Airflow / Temporal	observability
 Event-driven / reactive (2015)	Avro / Protobuf	Schema Registry	Kafka choreography	distributed tracing
 Cloud-native / serverless (2018)	OpenAPI / gRPC	Service mesh	K8s + Argo	OpenTelemetry
 Micro-agents (2024)	capability card / agent card	agent registry	planner / orchestrator	evals + traces

current unit:
probabilistic
capability



Same responsibilities; different unit behind the contract.

Session 3 · Recognise the shape

The same AOA shape in the wider landscape: frameworks, platforms, standards, registries, sovereignty.

Key notes

- **AOA's 25-year lineage.** CORBA → SOA → REST → microservices → event-driven → cloud-native → micro-agents. Architecture absorbs what is proven to work. AOA is continuation, not a break.
- **Frameworks live inside AUs.** LangGraph, CrewAI, AutoGen, EVE — implementation choices, not architectural boundaries. The architectural boundary is the AU contract. EVE is the strongest current interior — durable sessions, HITL, native evals — and still has no card, no registry, no identity: strong P1/P4 substrate, P3 enabler, not a P2 composition mechanism.
- **Three species, different altitude.** Orchestration frameworks (lowest), personal agents (middle), AOA fleets (highest). The right diagnostic question depends on which species you're looking at.
- **A2A outward + MCP inward.** Standards make Session 2's boundary portable across vendors. A2A is at v1.0: three equal bindings (JSON-RPC / gRPC / HTTP+JSON), signed agent cards, per-request auth — and platform implementations commonly lag the spec, so ask which version. Platforms are *pattern sources*, not rivals (see the platforms page overleaf); AOA is principles, not a product.
- **Registry governance is lifecycle, not a phone book.** Publish → approve → discover → observe → deprecate → withdraw. Without governance, the registry mis-leads selection — UDDI's failure mode in the agent era.
- **Sovereignty is layered.** Data · Model · Infrastructure · National policy — four independent axes, each with its own question and failure mode. *Sovereignty washing* is any claim that doesn't say which layer.
- **The trust pressure test.** *Who is acting, on whose authority, and why should another team or organisation trust it?*

The capstone you did

You ran the red-flags agent through EVE's native runtime, found it invisible to the governed system (no published card, registry entry, governed identity, AOA trace boundary or outward agent face), added one context-specific constraint to its public contract, then switched the same behaviour files to the `aoa-eve` adapter and watched it register as a first-class AU. The full lab is `system/agents-eve/EXERCISE.md`; repeat it with `./scripts/session3-lab-native.sh` followed by `./scripts/session3-lab-wrap.sh`. EVE supplies authoring and runtime; AOA adds the cross-agent composition and governance surface. Either side can evolve without binding consumers to the framework inside.

Platforms are pattern sources

The take-home version of the live platforms map, with the detail a slide cannot carry. Read every platform with the same question: which AOA responsibilities does it discharge, and which remain yours to design?

AOA is principles, not a product. Platforms are pattern sources, not rivals — and not a leaderboard.

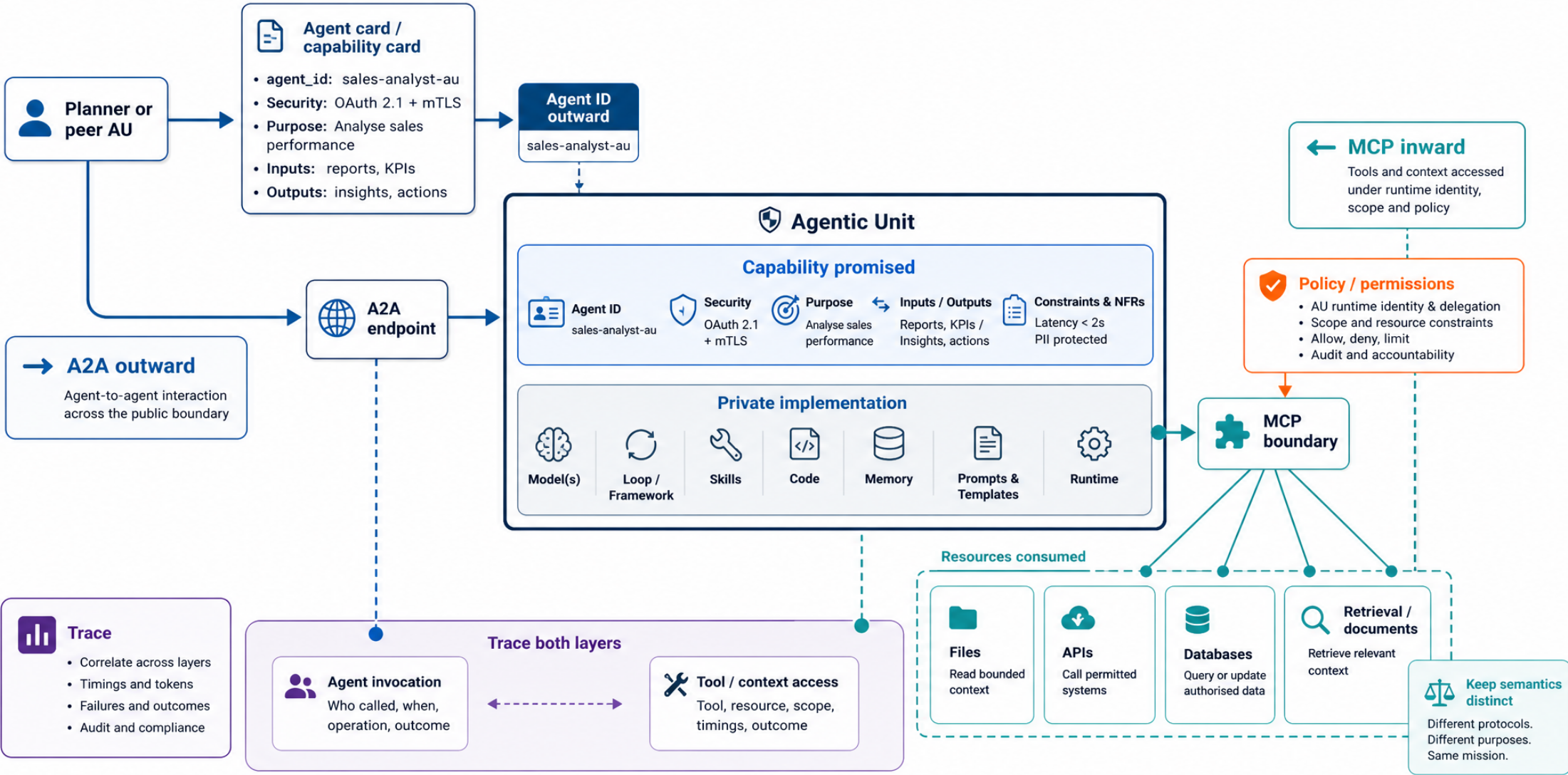
Platform	Pattern worth extracting	The unresolved AOA question
AWS AgentCore	Managed-stack convergence: runtime, registry, gateway, memory, identity, policy, observability, evaluation in one stack — strong signal these are becoming standard control surfaces	Can selection and replacement stay evidenced and cross-platform?
Google Gemini Enterprise / Vertex	Three AOA-novel patterns: SPIFFE per-agent IAM identity (each agent a first-class principal); bindings as a first-class registry resource (scoped, policy-gated source→target edges); a gateway that default-blocks unregistered targets — the closest production "registry as control plane"	Uniform policy across MCP / A2A / REST? Binding portability beyond Google?
Azure Foundry + Entra	The split-registry reality: Foundry runtime, Entra Agent Registry (identity), API Center (MCP tools), M365 Agent Store (consumer discovery) — "the registry" may be several coordinated surfaces	Portable beyond the Microsoft boundary? Cross-registry coordination?
Salesforce + MuleSoft	The agent-fabric pattern: catalogue/discovery, brokered orchestration, and gateway-layer policy as distinct layers	Ownership outside the platform estate?
ServiceNow AI Control Tower	Governance front and centre: agents, models, workflows, skills, prompts, datasets as governed assets; the AI Steward role makes lifecycle accountability explicit	Third-party lifecycle? Observed quality?
Camunda / BPM	Process orchestration with agents: deterministic joins, human approvals, audit trails — strong for compliance-heavy workflows	Is composition design-time process wiring, or contract-led discovery and replacement at runtime?

Permission to stay where you are

If your organisation is on AWS, Azure, Salesforce, ServiceNow, or Camunda, the question is not "throw it away." It is: which AOA responsibilities does your platform **discharge**, which does it **hide**, and which do you still have to **design**?

The absorption question

Pattern extraction tells you what a platform discharges. *Absorption* asks how to bring a third-party agent (vendor, partner, OSS) into your estate without inheriting its gaps: a code-level wrapper that normalises shape, a gateway-level proxy that enforces auth / redaction / observability, and an AOA **proxy agent card** registered first-class - declaring the gaps the proxy fills, the skills it refuses to forward, and the trust-class transformation it performs. Without that card, "we proxy our vendors" degenerates into shadow infrastructure the registry cannot reason about. (You built the same boundary yourself in the Session 3 capstone - the `aoa-eve` adapter is a proxy pattern in miniature.)



✓ **A2A carries identified peer capability work. MCP carries scoped tool and context access.**

Protocols and portability

Standards carry boundaries; they do not supply the architecture around them.

The placement rule

Direction	Protocol role	Architectural question still yours
A2A outward	Agent-to-agent discovery, cards, tasks, messages, and results	Who may publish, approve, invoke, select, and replace?
MCP inward	A governed AU invoking tools, resources, and prompts	What may this AU access, under whose authority, with what audit trail?

A2A v1.0 defines JSON-RPC, gRPC, and HTTP+JSON bindings. Shipping platform implementations may lag the specification, so record the protocol version and binding on the public surface rather than saying merely "A2A compatible". The companion `aoa-course` adapters currently emit the v0.3 JSON-RPC card and message shape. Use the lab to study boundary placement, not as a v1 conformance example.

The anti-lock-in test

A replacement is credible only when:

1. Consumers depend on the capability contract, not framework classes.
2. Identity and policy are outside the model/runtime implementation.
3. Traces and evaluation signals survive a provider change.
4. Registration changes the selected candidate without consumer rewrites.
5. A gateway or adapter does not hide undeclared semantic differences.

Portability is not "we can call another API". It is the ability to change the interior while preserving a governed promise and making behavioural differences observable.



Framework inside. Contract outside. Consumers do not inherit the framework choice.

Vendor tool inside, public contract outside

The EVE lab makes framework portability concrete without installing Node on the host.

Repeat the lab

```
./scripts/session3-lab-native.sh
# inspect the native EVE runtime, then stop the stack
./scripts/down.sh

./scripts/session3-lab-wrap.sh
# inspect the registry and Agent Card
./scripts/down.sh
```

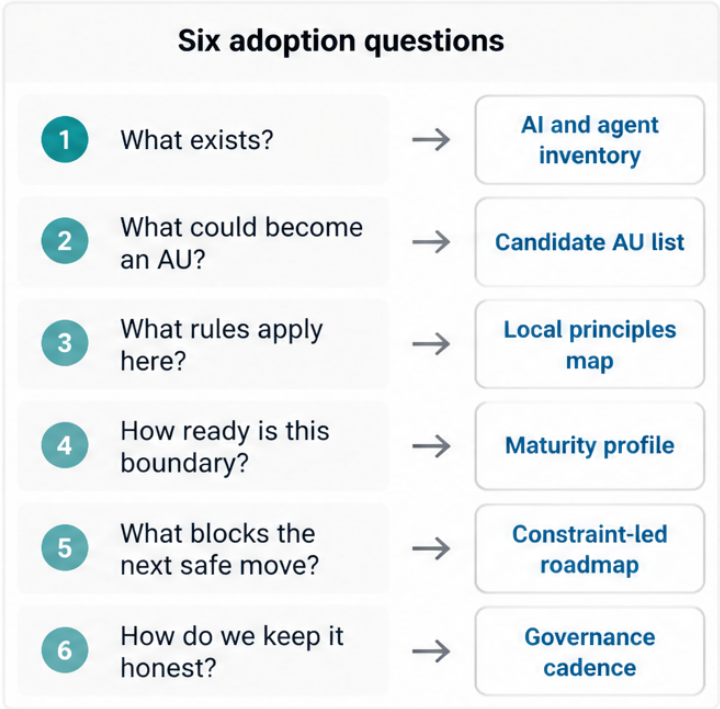
The pinned Docker image owns Node, EVE, and package dependencies. Both modes use the checked-in `agent.ts` and `instructions.md`.

Native EVE supplies	AOA adapter adds
Filesystem-first authoring	Capability card and stable Agent ID
Durable sessions and HITL	Registry publication and lifecycle
Tools, connections, sandbox	Outward A2A-compatible face
Native eval and runtime history	Cross-estate trace boundary and composition surface

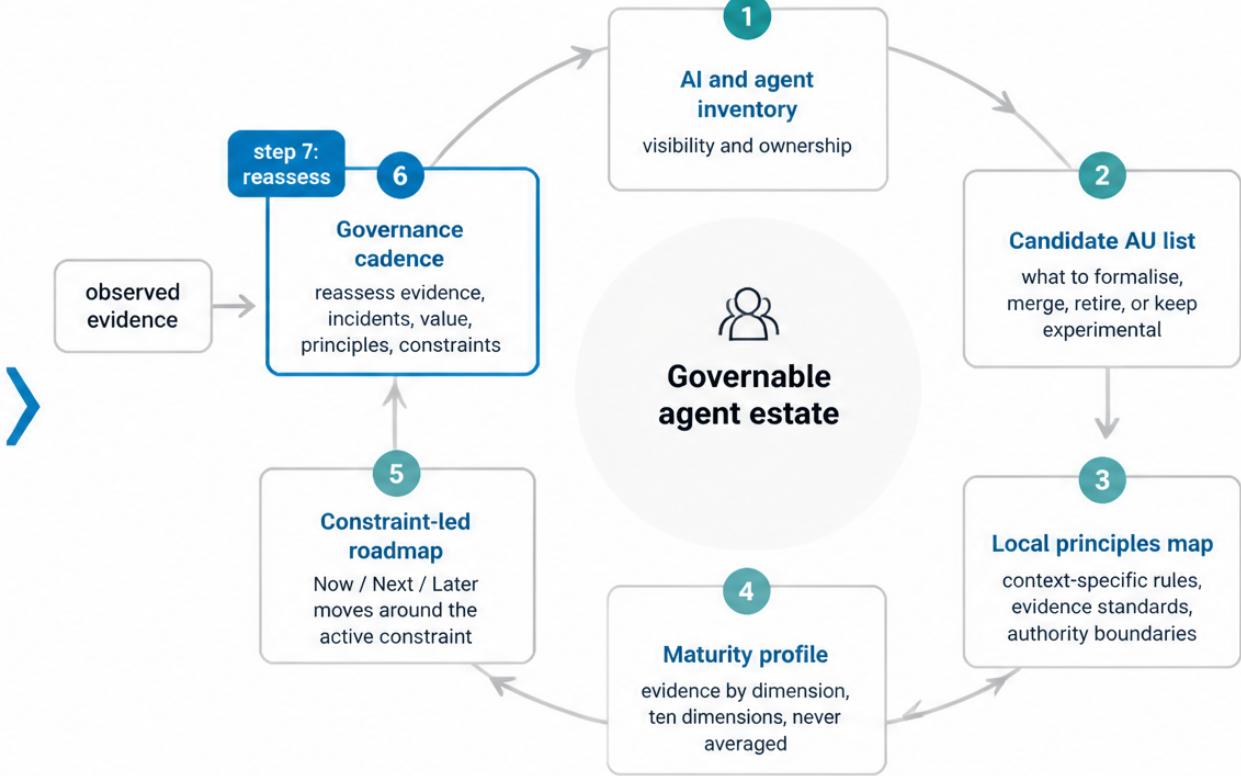
Add one constraint that is specific to your context, then ask: did behaviour change, or did the public promise become more explicit? The point is not that EVE is incomplete. It is that a strong vendor runtime and a vendor-neutral architecture can own different responsibilities.

Replacement test

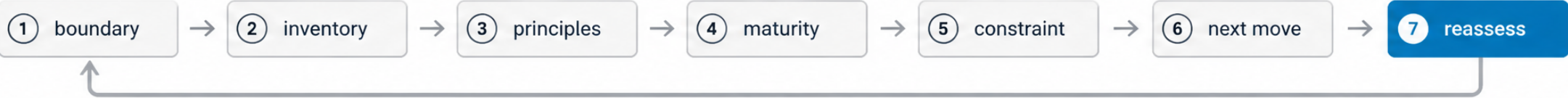
Could another runtime honour the same card, identity, inputs, outputs, constraints, and evidence expectations? If callers would need to import EVE types or understand its session model, the boundary still leaks.



Principles are localised here, then applied to candidate AUs.



Make the organisation more honest before making it more automated.



Session 4 · Apply the shape

Turn the architecture into a contextual adoption loop you can run on Monday.

Key notes

- **The estate checks itself.** The demo scans the day's registry and traces for evidence hooks related to selected EU AI Act obligations. The synthetic CV-fit exercise makes no hiring decision; a real employment deployment would be an Annex III candidate requiring contextual legal assessment. Findings carry verbatim regulation citations; green means *evidence present*, never "obligation satisfied". Re-run it: `./scripts/session4-up.sh + ./scripts/session4-seed.sh`.
- **A working tool may still be impermissible.** Architecture can expose cards, traces, oversight points, and lifecycle evidence. It cannot classify a deployment, grant permission to use it, or replace legal and risk judgement.
- **The demo is not step one.** Different starting points need different first moves. AOA is principles, not a product; copy the architecture only where the pressure exists.
- **Maturity is a contextual control loop, not a ladder.** The question is not "*what AOA maturity level are we?*" but "*which capabilities are mature enough for the next adoption move in this context — and which weak dimension is the active constraint?*"
- **The seven-step loop.** *Boundary → inventory → principles → maturity → constraint → next move → reassess.* Run the loop on a specific boundary, not the whole organisation.
- **Profile the boundary, not the org.** Ten dimensions, scored separately, **never averaged**. The weak spoke that the next move requires is the active constraint.
- **Move the active constraint.** *More throughput at the constraint, less WIP, better flow* — not "more agents." Theory of Constraints applied to AOA adoption. Identify → exploit → subordinate → elevate → repeat.
- **Five prioritisation filters: Value · Readiness · Control · Learning · Reuse.** The next move sits where all five overlap. *Now / Next / Later*, not calendar dates.
- **Six artefacts of a governable agent estate.** AI and agent inventory · AU candidate list · local principles map · maturity profile · constraint-led roadmap · governance cadence.



EU AI Act — Regulation 2024/1689

original schedule: Aug 2026



current Commission guidance: 2 Dec 2027



real employment use: Annex III candidate — contextual legal assessment required

The Act asks for

Art 9 risk management system

Art 10 data governance

Art 11 technical documentation

Art 12 record-keeping / logging

Art 13 transparency

Art 14 human oversight

Art 72 post-market monitoring

Possible AOA evidence hooks



evaluation portfolio + replacement path



declared tool & data boundary

 partial



capability card + Agent ID



the trace



card: declared constraints & signals



human oversight point



registry lifecycle + observed signals



AOA exposes evidence hooks. It does not grant permission to deploy.

A working tool may still be impermissible

The Session 4 story is a boundary on architecture's authority, not a legal footnote.

What the course can safely claim

- The classroom workflow uses synthetic CVs and makes no employment decision.
- A real use intended to analyse, filter, or evaluate candidates matches the language of Annex III point 4 and requires contextual legal assessment, including Article 6(3).
- The Regulation's original schedule pointed to August 2026. Following the 7 May 2026 AI Omnibus political agreement, current European Commission guidance says specified high-risk areas including employment apply from **2 December 2027**. Verify the enacted text and current guidance before any deployment.
- A capability card, trace, oversight point, or lifecycle event is an evidence hook. Its presence does not prove an obligation is satisfied.

The seven review lenses

AI Act area	Possible AOA evidence hook	What remains outside
Art 9 risk management	Evaluation portfolio, replacement path	Complete risk process and effectiveness
Art 10 data governance	Declared input/tool boundary	Dataset quality, representativeness, lawful processing
Art 11 technical documentation	Card, Agent ID, provenance	Required completeness and conformity evidence
Art 12 record-keeping	Trace and retention policy	Required event coverage and lawful retention
Art 13 transparency	Declared constraints and signals	Adequacy for the real deployer and use
Art 14 human oversight	Escalation and approval points	Competence, authority, usability, effectiveness
Art 72 post-market monitoring	Lifecycle and observed signals	Legally sufficient monitoring plan and incident process

Sources: [European Commission AI Act timeline](#) and [high-risk-system guidelines](#), updated July 2026.

Internal research workflow

evidence & disagreement



weakest area: governance

candidate
constraint
governance

Regulated customer decision

evidence & disagreement



weakest area: evaluation

candidate
constraint
evaluation

Engineering automation

evidence & disagreement



weakest area: visibility

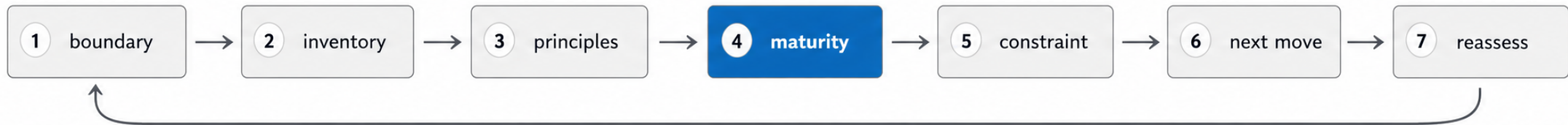
candidate
constraint
visibility

1 ad-hoc · 2 emerging
3 defined · 4 managed
5 adaptive

never average — the
weak spoke is the signal

strategy · visibility · AU definition · discovery · standards · context · orchestration · evaluation · governance · people

No single score. Profile the boundary. Find the unevenness.



Adoption canvas

Profile a boundary, not the organisation. Never average the dimensions.

Fill this in with the people who own the boundary

Decision	Your answer
Boundary: domain, value stream, product, or team	
Business outcome and current pain	
Existing agents, copilots, workflows, and tools	
Data, authority, and irreversible side effects	
Named business and technical owners	
Weakest maturity dimension required by the next move	
Evidence for that assessment	
Active constraint	
Smallest move that increases learning and control	
Signal that tells us the constraint moved	
Reassessment date and owner	

Ten dimensions

Strategy · visibility · AU definition · discovery · integration · context · runtime · evaluation · governance · people.

Use disagreement as data. A low score with evidence is more useful than a high average that hides a blocking dimension. Prioritise the next move through five filters: **Value · Readiness · Control · Learning · Reuse.**

Monday morning

What to do when the course is over.

The seven-step loop in one sentence

Pick a boundary. Inventory what exists. Localise the principles. Build a maturity profile. Find the active constraint. Make the next move. Reassess when the constraint moves.

The six artefacts of a governable agent estate

1. **AI and agent inventory** — what exists, who owns it, what it touches, lifecycle state.
2. **AU candidate list** — what to formalise, merge, retire, or keep experimental.
3. **Local principles map** — generic AOA principles + your context = your rules of adoption.
4. **Maturity profile** — ten dimensions, scored separately, with evidence and disagreement captured.
5. **Constraint-led roadmap** — Now / Next / Later, around the active constraint, filtered by Value × Readiness × Control × Learning × Reuse.
6. **Governance cadence** — owners review evidence, incidents, value, principles alignment, and whether the constraint has moved.

How to draw an AU boundary — the six-step recipe

Two questions have drawn boundaries since 1972: what design decision does this unit hide — and who is entitled to ignore it? and what is its one dominant reason to change?

1. **Name the capability in the business's language** — an unrecognisable name is an implementation detail.
2. **List the decisions it hides** — an empty list means it's a step, not a unit.
3. **Find the one dominant reason to change** — three unrelated reasons are three units.
4. **Write the contract before the prompt** — the prompt is private; the contract is public.
5. **Classify on two axes** — context-bound / context-free × judgement / transformation → *domain AU · domain tool · utility AU · utility tool*. This decides evaluated-vs-tested, A2A-vs-MCP, governance weight.
6. **Run the 10-question boundary test** — the gate, not the starting point.

The honest closing claim

An AOA adoption roadmap should make the organisation more honest before it makes it more automated. It should reveal what AI and agentic capability already exists, where ownership is unclear, which work should be redesigned, which standards are useful, which principles are non-negotiable, and which maturity gaps would make orchestration unsafe.

Where to go next

- **The running system:** `aoa-course` companion repo at `localhost:8080` — three agents, capability cards, registry view, browser studio with the trace panel.
- **Contact:** `lewis.crawford@equalexports.com`
- **LinkedIn:** [linkedin.com/in/lewiscrawford](https://www.linkedin.com/in/lewiscrawford)

Compact reference

Core vocabulary

Term	Working definition
Agentic Unit (AU)	A bounded, independently governable capability with a public contract and model-driven judgement inside
Capability card	The public promise: purpose, I/O, constraints, evidence expectations, provenance, and lifecycle metadata
Agent ID	Stable governed identity of the acting unit; a hash is an integrity signal, not identity
Registry	Discovery plus lifecycle: publish, review, approve, observe, deprecate, replace, withdraw
Planner	Proposes a composition by reading intent and capability contracts
Orchestrator	Executes the plan, carries state, enforces policy, and records the trace
Tool	A deterministic capability invoked behind a declared boundary, commonly through MCP
Trace	Correlated runtime evidence across selection, invocation, tools, outputs, signals, and escalation
Evaluation signal	A measurable observation that can challenge the card's claims
Oversight point	A declared boundary where human judgement, approval, or escalation is required

Questions to take into an architecture review

1. What is this unit's one capability and one dominant reason to change?
2. Which decisions and authority does it own, and which are explicitly denied?
3. What do consumers depend on that is independent of framework and model?
4. Who may publish, approve, invoke, observe, deprecate, and replace it?
5. Which tools and data may it access, on whose behalf?
6. What evidence would falsify its quality, cost, latency, or safety claims?
7. Where are silent recovery and explicit escalation each allowed?
8. Can the trace identify the human, agent, model, tools, and policy involved?
9. Can another implementation honour the same public contract without consumer changes?
10. What external legal, risk, or organisational judgement cannot be delegated to the architecture?

The closing test

Can you describe, discover, compose, execute, observe, evaluate, replace, and govern the capability without knowing which framework is inside it?

If not, the architecture boundary is still inside the implementation.